

LABORATORY 2: Network Communication & The Client-Server Model

1. Laboratory Objectives

By the end of this laboratory, students will be able to:

1. **Implement** a basic TCP client–server architecture using the Socket API.
2. **Differentiate** between an iterative (blocking) server and a concurrent (threaded) server.
3. **Observe** head-of-line blocking, where one slow client affects all other clients.
4. **Apply** thread creation techniques (from Lab 1) to handle network connections dynamically.
5. **Analyze** the fundamental trade-off between resource consumption (threads) and responsiveness (latency).
6. **Implement** a thread-safe task queue to support concurrency and handle multiple client/worker connections.
7. **Design** an application-layer protocol over TCP sockets.
8. **Compare** TCP, UDP, and QUIC in terms of reliability, ordering, congestion control, and latency.

Lab resources:

https://drive.google.com/drive/folders/175jG_WDyRPF3CaWz-XfMF-kaekbbGqH5?usp=drive_link

2. Theoretical Background

2.1. Message Passing vs. Shared Memory

In Lab 1, threads communicated by reading/writing to the same memory address. In Network Programming, processes (even on the same machine) do not share memory. They communicate by sending streams of bytes (Messages) through the OS kernel via **Sockets**.

2.2. The Socket API Lifecycle

A Socket is a file descriptor used for network I/O.

- **Server:** `socket()` -> `bind()` (reserve port) -> `listen()` (wait) -> `accept()` (handshake).
- **Client:** `socket()` -> `connect()` (initiate handshake).

2.3. Blocking I/O

Standard socket calls (`recv`, `accept`) are **blocking**.

- If the server calls `recv()` and the client hasn't sent data yet, the server **sleeps**.
- In a single-threaded (Iterative) server, while the server sleeps waiting for Client A, Client B cannot connect. This is the bottleneck we must solve.

2.4. TCP (Transmission Control Protocol)

A connection-oriented, reliable, stream-based protocol. Guarantees in-order delivery through sequence numbers, acknowledgments, and retransmission. Establishes connections via a 3-way handshake (SYN, SYN-ACK, ACK) and tears them down with a 4-way close (FIN/ACK). Provides flow control (sliding window) and congestion control (slow start, congestion avoidance).

Use when: reliability matters more than latency -- file transfers, web pages, database queries, the Coordinator-Producer-Worker system in this lab.

Example: HTTP/1.1, SSH, SMTP, PostgreSQL wire protocol.

```
// Minimal TCP client (C, POSIX)
int sock = socket(AF_INET, SOCK_STREAM, 0);
connect(sock, (struct sockaddr*)&addr, sizeof(addr)); // 3-way handshake
send(sock, data, len, 0); // reliable, ordered delivery
recv(sock, buf, buflen, 0); // stream-based (no message boundaries)
close(sock); // 4-way teardown
```

References:

- RFC 9293 -- TCP specification (2022, replaces RFC 793): <https://www.rfc-editor.org/rfc/rfc9293>
- Beej's Guide to Network Programming (practical C examples): <https://beej.us/guide/bgnet/>
- Linux tcp(7) man page: <https://man7.org/linux/man-pages/man7/tcp.7.html>

2.5 UDP (User Datagram Protocol)

A connectionless, unreliable, message-based protocol. No handshake, no acknowledgments, no ordering guarantees. Each `sendto()` maps to exactly one datagram; messages have clear boundaries (unlike TCP's byte stream). Minimal overhead (8-byte header vs. TCP's 20+ bytes).

Use when: low latency matters more than reliability -- real-time audio/video, DNS lookups, game state updates, IoT telemetry. Applications implement their own reliability if needed.

Example: DNS, DHCP, RTP (voice/video), game netcode.

```
// Minimal UDP sender (C, POSIX)
int sock = socket(AF_INET, SOCK_DGRAM, 0); // No connect() needed -- connectionless
sendto(sock, data, len, 0, (struct sockaddr*)&addr, sizeof(addr)); // fire and forget
recvfrom(sock, buf, buflen, 0, NULL, NULL); // one recv = one datagram
close(sock);
```

References:

- RFC 768 -- UDP specification (the entire spec is 3 pages): <https://www.rfc-editor.org/rfc/rfc768>
- Linux udp(7) man page: <https://man7.org/linux/man-pages/man7/udp.7.html>

2.6 QUIC (Quick UDP Internet Connections)

A modern transport protocol built on top of UDP, designed by Google and standardized by IETF in 2021. Combines the reliability of TCP with the low-latency connection setup of UDP. Key innovations:

0-RTT / 1-RTT connection setup: merges the transport handshake with TLS 1.3 encryption in a single round-trip (TCP+TLS requires 2-3 round-trips).

Stream multiplexing without head-of-line blocking: multiple independent streams over one connection. If one stream's packet is lost, other streams continue (unlike TCP, where a single lost packet stalls everything).

Connection migration: connections survive IP address changes (e.g., switching from Wi-Fi to cellular) because connections are identified by a Connection ID, not by IP:port tuple.

Built-in encryption: TLS 1.3 is mandatory; there is no unencrypted QUIC.

Use when: modern web applications, mobile-first apps, anything that needs multiplexed streams with encryption -- HTTP/3 runs entirely over QUIC.

Example: HTTP/3 (all major browsers), Google services, Cloudflare edge network.

- Note: QUIC is a userspace protocol (runs over UDP port 443), not a kernel-level protocol like TCP/UDP. It cannot be used with raw socket() calls -- it requires a library.

Libraries:

- quiche (Cloudflare, Rust/C): <https://github.com/cloudflare/quiche>
- ngtcp2 (C): <https://github.com/ngtcp2/ngtcp2>
- msquic (Microsoft, C): <https://github.com/microsoft/msquic>
- aioquic (Python): <https://github.com/aiortc/aioquic>

References:

- RFC 9000 -- QUIC Transport Protocol: <https://www.rfc-editor.org/rfc/rfc9000>
- RFC 9001 -- Using TLS to Secure QUIC: <https://www.rfc-editor.org/rfc/rfc9001>
- RFC 9114 -- HTTP/3 (over QUIC): <https://www.rfc-editor.org/rfc/rfc9114>
- QUIC Working Group (IETF): <https://quicwg.org/>

- Cloudflare "The Road to QUIC" (accessible overview):
<https://blog.cloudflare.com/the-road-to-quic/>

2. 7 Quick Comparison

	TCP	UDP	QUIC
Connection	3-way handshake	None	1-RTT (0-RTT for repeat)
Reliability	Yes (retransmit)	No	Yes (per-stream)
Ordering	Yes (global)	No	Yes (per-stream)
Encryption	Optional (TLS)	Optional (DTLS)	Mandatory (TLS 1.3)
Multiplexing	No (1 stream)	N/A	Yes (no HOL blocking)
Header size	20+ bytes	8 bytes	Variable (~16+ bytes)
Runs on	IP (kernel)	IP (kernel)	UDP (userspace)
Standard	RFC 9293	RFC 768	RFC 9000

3. Laboratory Activities

Exercise 1: The "Lazy" Iterative Server

Goal: Observe the limitations of a single-threaded server.

1. **Code (Provided in Appendix):** Compile the basic `server_iterative.c`. (the code is in Appendix)
2. **Action:** The server accepts a connection, reads a message, waits 5 seconds (simulating heavy processing), echoes the message back, and closes.
3. **Experiment:**
 - Open Terminal 1: Run Server.
 - Open Terminal 2: Run Client A (`nc localhost 8080`). Send "Hello".
 - *Immediately* Open Terminal 3: Run Client B (`nc localhost 8080`). Send "Hi".

Analysis Question:

What happens to Client B? Does it get a response immediately?

Why does Client B have to wait for Client A to finish, even though they are independent? (Explain "Head-of-Line Blocking").

Exercise 2: The Concurrent Server (Thread-per-Client)

Goal: Use concurrency to handle multiple clients simultaneously.

1. **Task:** Modify the `server_iterative.c` code to become `server_concurrent.c`.
2. **Logic:**
 - The `main` loop should only do `accept()`.
 - Immediately after accepting a connection (getting a `client_fd`), spawn a **new thread** (`pthread_create`).
 - The new thread executes the `handle_client` function.
 - **Important:** Don't forget to detach the thread (`pthread_detach`) to avoid memory leaks (zombie threads).

Analysis Question:

Run the same experiment as Ex 1 (Client A and Client B).

Does Client B still wait for Client A?

Thinking: What happens if 10,000 clients connect at the exact same second? What is the risk of "Thread-per-Client"? (Context Switch Overhead, Out of Memory).

Exercise 3: Simulating Network Unreliability

Goal: Understand that network writes are not instant.

1. Modify the Client to send a large amount of data (e.g., a 10MB file).
2. Introduce a `sleep(1)` inside the server's `recv/read` loop.
3. Observe that `send()` on the client side might return success *before* the server has actually processed the data.

Theory Check:

When `send()` returns, has the data reached the other application?

Answer: No. It has only been copied to the OS Kernel's TCP Send Buffer.

Objective: Demonstrate that when a generic `send()` function returns `Success`, it **does not** mean the data has reached the destination application. It only means the data has been copied to the Operating System's Kernel Buffer.

The Experiment:

1. **Fast Client:** Sends a large volume of data instantly and exits. (`client_fast.c`)
2. **Slow Server:** Reads data byte-by-byte with a delay. (`server_slow.c`)

How to Analyze

1. Run `./server_slow`.
2. Run `./client_fast`.
3. **Observation:** The Client says "Time taken: 0.000... seconds" and exits immediately. However, the Server will keep printing "Received chunk..." for almost a minute (due to `sleep(1)`).
4. **Conclusion:** The data was safely sitting in the **TCP Send Buffer** of the Client OS and the **TCP Receive Buffer** of the Server OS, not in the application.

4. LabWork: The Thread Pool Server

Context: Creating a thread for every client (Ex 2) is dangerous for scale. Real systems (like Nginx or Apache) use a **Thread Pool**.

Task:

1. Create a fixed pool of worker threads (e.g., 4 threads) at startup.
2. Implement a **Task Queue** (using a Linked List or Array).
3. The `main` thread accepts connections and adds the `client_socket` to the Queue.
4. Worker threads sleep until the Queue has an item (Producer-Consumer problem using Condition Variables - linking back to Lab 1).

Why this matters: This architecture prevents the server from crashing under load; it simply processes requests slower when the queue fills up, providing **Backpressure**.

Objective: Implement a production-ready server architecture using the **Producer-Consumer** pattern. Instead of creating a thread for every client (which crashes the OS at scale), we use a fixed number of threads consuming tasks from a queue.

Compilation & Testing

1. **Compile:**
 - a. Compile the Thread Pool Server: `gcc -o server_pool server_threadpool.c -pthread`
 - b. Compile this Stress Test Client: `gcc -o stress_test client_stress_test.c -pthread`
2. **Run the Experiment:**
 - a. **Terminal 1:** Start the server: `./server_pool`
 - b. **Terminal 2:** Run the stress test: `./stress_test`
3. **Expected Output & Analysis**
 - a. **On the Client Side (`./stress_test`):** You will see 20 lines appearing very quickly.
 - i. [Client 0] Server replied: Processed by ThreadPool

- ii. [Client 1] Server replied: Processed by ThreadPool
- iii. ...
- b. **On the Server Side (./server_pool)**: This is where the magic happens. Even though 20 clients arrived instantly, you will see the **Task Queue** logic in action:
 - i. You will see [Main] accepting connections rapidly and calling enqueue.
 - ii. You will see the 4 Worker Threads picking up tasks.
 - iii. **Key Observation**: Since there are only 4 threads, the logs will show them being reused.
 - 1. Example: [Worker Thread 12345] handles socket 5.
 - 2. Then [Worker Thread 12345] handles socket 9.
 - iv. If the clients were real users, the last few clients would experience a slightly higher latency than the first few, because they sat in the queue waiting for a worker to become free.

5. QUIC (msquic, quiche)

msquic

```
# --- SETUP INSTRUCTIONS ---
# 1. Install msquic runtime:
#   curl -sSL https://packages.microsoft.com/keys/microsoft.asc | sudo tee
/etc/apt/trusted.gpg.d/microsoft.asc
#   sudo apt-add-repository https://packages.microsoft.com/ubuntu/$(lsb_release -rs)/prod
#   sudo apt update
#   sudo apt install libmsquic
#
# 2. Build (headers are downloaded automatically):
#   make
#
# 3. Run (3 terminals):
#   Terminal 1: ./quic_coordinator
#   Terminal 2: ./quic_producer 5
#   Terminal 3: ./quic_worker
```

quiche

```
#
=====
# SETUP INSTRUCTIONS
#
=====
# 1. Install Rust (if not already):
#   curl --proto 'https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
#   source ~/.cargo/env
#
# sudo apt install build-essential cmake pkg-config libclang-dev libfontconfig1-dev
```

```
#
# 2. Build everything (clones quiche, compiles Rust lib, compiles C code):
#   make
#
# 3. Run (3 terminals):
#   Terminal 1: ./quiche_coordinator
#   Terminal 2: ./quiche_producer 5
#   Terminal 3: ./quiche_worker
#
# Notes:
# - First build takes 2-5 minutes (Rust compilation + BoringSSL).
# - Subsequent builds are fast (only C code recompiles).
# - `make distclean` removes the quiche-repo entirely.
# - The static library (libquiche.a) bundles BoringSSL, so no
#   separate OpenSSL/BoringSSL linkage is needed at runtime.
```

6. UDP

(see udp folder on Google Drive)

7. Coordinator-Producer-Worker

Coordinator (Broker): maintains a task queue, assigns tasks to ready workers, receives DONE reports.

Producer (Client): submits tasks (**SUBMIT** ...) from stdin or auto-generated.

Worker (Consumer): connects, requests work (**READY**), simulates processing (**sleep**), reports back (**DONE**).

https://drive.google.com/drive/folders/16QgmW7ImWIsHeGR_zc2DuhsFmYvnZgh1?usp=drive_link
